

INFOSYS LIMITED

NeoBank 360

Software Handover & Architecture Document

Project Name	NeoBank 360
Delivery Unit / Location	Infosys Bhubaneswar Development Center
Lab Allocation	Lab 4 & Lab 5
Program	PMIS Internship Program
Team Members	<i>[Insert Here]</i>
Document Version	<i>[Insert Here]</i>
Date of Submission	<i>[Insert Here]</i>
Classification	Confidential – Internal Use Only

Document Control

Revision History

Version	Date	Description of Change	Author	Approved By
1.0	<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>

Distribution List

This document is intended for circulation to the following stakeholders. Recipients must treat the contents as confidential and refrain from external distribution without prior written authorization from Infosys Limited.

- Project Sponsor / Engagement Manager: *[Insert Here]*
- Technical Lead / Architect: *[Insert Here]*
- Quality Assurance Lead: *[Insert Here]*
- PMIS Internship Program Coordinator: *[Insert Here]*

Table of Contents

Document Control	2
Revision History	2
Distribution List.....	2
Table of Contents.....	3
1. Executive Summary	5
1.1 Project Snapshot	5
1.2 Objectives.....	5
1.3 Scope of This Document	5
2. System Architecture	6
2.1 High-Level Architecture Overview.....	6
2.2 Entity-Relationship (ER) Diagram	6
2.3 N-Tier Architecture Flow	6
3. Core Modules Overview	7
3.1 Inter-Module Dependencies.....	8
4. Security & Authentication	8
4.1 Authentication Mechanism (JWT)	8
4.2 Authorization Model (RBAC).....	9
4.3 Additional Security Controls.....	9
5. Performance Optimizations	9
5.1 Caching Strategy	9
5.2 Database Indexing Strategy.....	10
5.3 Additional Performance Measures.....	10
6. Deployment Guide	10
6.1 Prerequisites	10
6.2 Containerization (Dockerfile).....	11
6.3 Environment Variables.....	11
6.4 Deployment Steps	11
6.5 Rollback Procedure	12
7. Testing Summary	12
7.1 End-to-End (E2E) Testing.....	12
7.2 Load & Performance Testing	12
7.3 Testing Tools & Environment.....	13
8. Challenges & Learnings	13
8.1 Technical Challenges Encountered	13
8.2 Key Learnings.....	13

8.3 Recommendations for Future Enhancements13

Annexure A: Glossary of Terms14

Document Sign-Off14

1. Executive Summary

This document constitutes the formal Software Handover and Architecture record for NeoBank 360, prepared as part of the PMIS Internship Program executed at the Infosys Bhubaneswar Development Center, Lab 4 & 5. It is intended to provide senior stakeholders, incoming engineering teams, and quality assurance personnel with a complete and authoritative reference covering the system's architecture, core functional modules, security posture, performance engineering, deployment procedure, and validation outcomes.

NeoBank 360 has been engineered as a unified digital banking platform that consolidates account management, budgeting, bill management, lending, and analytical capabilities within a single, secure, and horizontally scalable application. The purpose of this handover is to ensure continuity of ownership, reduce onboarding time for new contributors, and establish a single source of truth for all architectural and operational decisions taken during the development lifecycle.

1.1 Project Snapshot

Attribute	Details
Project Name	NeoBank 360
Domain	<i>[Insert Here]</i>
Technology Stack	<i>[Insert Here]</i>
Team Size	<i>[Insert Here]</i>
Project Duration	<i>[Insert Here]</i>
Delivery Status	<i>[Insert Here]</i>
Hosting Environment	<i>[Insert Here]</i>

1.2 Objectives

- Primary Business Objective: *[Insert Here]*
- Target User Base: *[Insert Here]*
- Key Differentiators: *[Insert Here]*

1.3 Scope of This Document

The scope of this handover document encompasses the application architecture, the six core functional modules, the security and authentication framework, performance optimization strategies, the containerized deployment process, and a summary of testing outcomes. It excludes day-to-day operational runbooks and detailed API contracts, which are maintained separately in the project's technical wiki and source code repository.

2. System Architecture

NeoBank 360 is designed around a layered, N-Tier architectural pattern that separates presentation, business logic, and data persistence concerns. This separation enables independent scaling, simplified maintenance, and a clear division of responsibility across the development team. The subsections below provide the visual and descriptive architecture artifacts required for technical due diligence.

2.1 High-Level Architecture Overview

Provide a narrative summary of the overall architectural style adopted (e.g., monolithic, modular monolith, or microservices), the rationale behind the chosen approach, and how it supports the scalability and security requirements of a financial application.

Architecture Style & Rationale

Describe the chosen architectural pattern, key design decisions, and supporting rationale.

[Insert Here]

2.2 Entity-Relationship (ER) Diagram

Insert the complete Entity-Relationship diagram representing the NeoBank 360 data model below. The diagram should clearly depict all entities, attributes, primary and foreign keys, and the cardinality of relationships across the Accounts, Transactions, Budgets, Bills, Loans, and Analytics domains.

ER Diagram – NeoBank 360 Data Model

Insert diagram image or embedded object here (e.g., exported from dbdiagram.io, MySQL Workbench, or Lucidchart).

[Insert Here]

2.3 N-Tier Architecture Flow

Insert the N-Tier architecture flow diagram illustrating request and data flow across the Presentation, API/Application, Business Logic, and Data Access tiers.

N-Tier Architecture Flow Diagram

Insert flow diagram depicting the request lifecycle from client to database and back.

[Insert Here]

The table below summarizes the responsibility and technology mapping for each architectural tier.

Tier	Responsibility	Technology / Framework
Presentation Layer	[Insert Here]	[Insert Here]
API / Gateway Layer	[Insert Here]	[Insert Here]
Business Logic Layer	[Insert Here]	[Insert Here]
Data Access Layer	[Insert Here]	[Insert Here]
Database / Persistence Layer	[Insert Here]	[Insert Here]

3. Core Modules Overview

NeoBank 360 is composed of six core functional modules, each independently testable and individually responsible for a distinct domain of banking functionality. The table below summarizes the purpose, key features, and underlying technology for each module. Development teams must populate the description, feature, and technology columns with implementation-accurate details prior to final handover sign-off.

Module	Description	Key Features	Technology / Notes
Auth	[Insert Here]	[Insert Here]	[Insert Here]
Accounts	[Insert Here]	[Insert Here]	[Insert Here]
Budgets	[Insert Here]	[Insert Here]	[Insert Here]
Bills	[Insert Here]	[Insert Here]	[Insert Here]
Loans	[Insert Here]	[Insert Here]	[Insert Here]

Module	Description	Key Features	Technology / Notes
Analytics	[Insert Here]	[Insert Here]	[Insert Here]

3.1 Inter-Module Dependencies

Describe how the modules interact with one another (e.g., the Analytics module consuming data from Accounts, Budgets, and Bills; the Loans module depending on Accounts for disbursement and repayment).

Module Dependency Map

Insert a diagram or description of inter-module data flow and dependencies.

[Insert Here]

4. Security & Authentication

Security is a foundational design pillar for NeoBank 360, given the sensitivity of financial data handled by the platform. The application enforces authentication via JSON Web Tokens (JWT) and authorization via Role-Based Access Control (RBAC). This section must be completed with implementation-specific details to enable accurate security review by senior stakeholders.

4.1 Authentication Mechanism (JWT)

Explain the end-to-end JWT authentication flow, including token issuance at login, token structure and claims, signing algorithm, expiry duration, and the refresh token strategy.

JWT Authentication Flow

Insert a sequence diagram or detailed explanation of the login, token issuance, and token refresh flow.

[Insert Here]

- Signing Algorithm: [Insert Here]

- **Access Token Expiry:** *[Insert Here]*
- **Refresh Token Expiry & Rotation Policy:** *[Insert Here]*
- **Token Storage Mechanism (Client-Side):** *[Insert Here]*

4.2 Authorization Model (RBAC)

Document the role hierarchy implemented within NeoBank 360 and the permissions associated with each role. The table below should be populated with the exact roles defined in the codebase.

Role	Permissions	Description
<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>
<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>
<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>

4.3 Additional Security Controls

- **Data Encryption (In-Transit / At-Rest):** *[Insert Here]*
- **Input Validation & Sanitization Strategy:** *[Insert Here]*
- **API Rate Limiting & Throttling:** *[Insert Here]*
- **Audit Logging & Monitoring:** *[Insert Here]*
- **Secrets Management Approach:** *[Insert Here]*

5. Performance Optimizations

To ensure responsiveness under production-grade load, NeoBank 360 incorporates targeted optimizations across the caching and database layers. This section captures the specific strategies implemented, enabling future contributors to understand and extend the system's performance profile without regression.

5.1 Caching Strategy

Describe the caching layer adopted (e.g., in-memory, distributed cache such as Redis), the data entities cached, time-to-live (TTL) configuration, and the cache invalidation strategy.

Caching Architecture

Insert details or diagram of the caching layer, cache keys, TTL policy, and invalidation triggers.

[Insert Here]

- Cache Provider / Technology: **[Insert Here]**
- Cached Entities: **[Insert Here]**
- Cache Invalidation Strategy: **[Insert Here]**

5.2 Database Indexing Strategy

Document the indexing strategy applied to high-traffic tables, including composite indexes, query patterns optimized, and any partitioning or sharding considerations.

Indexing & Query Optimization

Insert details of indexed columns, query execution plans, and partitioning strategy where applicable.

[Insert Here]

- Indexed Tables / Columns: **[Insert Here]**
- Query Optimization Techniques Applied: **[Insert Here]**

5.3 Additional Performance Measures

- Connection Pooling Configuration: **[Insert Here]**
- Pagination & Lazy Loading Strategy: **[Insert Here]**
- Asynchronous / Background Processing: **[Insert Here]**
- Observed Performance Benchmarks: **[Insert Here]**

6. Deployment Guide

This section provides the operational instructions required to build, containerize, and deploy NeoBank 360 across development, staging, and production environments. All commands and configuration values must be validated by the development team prior to handover.

6.1 Prerequisites

- Runtime Environment & Version: **[Insert Here]**
- Container Platform (e.g., Docker Engine Version): **[Insert Here]**
- Database Server & Version: **[Insert Here]**
- Required Third-Party Services: **[Insert Here]**

6.2 Containerization (Dockerfile)

Insert the production Dockerfile build and run commands used to containerize the NeoBank 360 application.

```
# Docker Build & Run Commands
```

```
[Insert Here]
```

6.3 Environment Variables

The following environment variables must be configured prior to deployment. Sensitive values must never be committed to source control and should be managed via a secrets manager or CI/CD secured variable store.

Variable Name	Description	Example Value
DB_HOST	[Insert Here]	[Insert Here]
DB_PORT	[Insert Here]	[Insert Here]
DB_NAME	[Insert Here]	[Insert Here]
JWT_SECRET	[Insert Here]	[Insert Here]
JWT_EXPIRY	[Insert Here]	[Insert Here]
PORT	[Insert Here]	[Insert Here]
NODE_ENV / SPRING_PROFILE	[Insert Here]	[Insert Here]
[Insert Here]	[Insert Here]	[Insert Here]

6.4 Deployment Steps

1. Pull the latest stable build from the source repository.
2. Configure environment variables as per Section 6.3.
3. Build the Docker image using the command specified in Section 6.2.
4. Run database migrations / schema initialization scripts. [Insert Here]
5. Deploy the container to the target environment (e.g., orchestration platform). [Insert Here]
6. Verify application health via the designated health-check endpoint. [Insert Here]

6.5 Rollback Procedure

Document the rollback strategy in the event of a failed deployment, including how to revert to the previous stable container image and database state.

Rollback Strategy

Insert rollback steps, including image versioning and database rollback considerations.

[Insert Here]

7. Testing Summary

Comprehensive testing was conducted across functional and non-functional dimensions to validate the correctness, reliability, and scalability of NeoBank 360 prior to handover. This section summarizes the End-to-End (E2E) and Load Testing outcomes.

7.1 End-to-End (E2E) Testing

Parameter	Details
Testing Tool / Framework	<i>[Insert Here]</i>
Test Coverage (% of Modules)	<i>[Insert Here]</i>
Total Test Cases Executed	<i>[Insert Here]</i>
Pass / Fail Summary	<i>[Insert Here]</i>
Key Scenarios Validated	<i>[Insert Here]</i>
Critical Defects Identified	<i>[Insert Here]</i>

7.2 Load & Performance Testing

Metric	Target / Threshold	Observed Result
Concurrent Users Simulated	<i>[Insert Here]</i>	<i>[Insert Here]</i>
Average Response Time	<i>[Insert Here]</i>	<i>[Insert Here]</i>

Metric	Target / Threshold	Observed Result
95th Percentile Response Time	<i>[Insert Here]</i>	<i>[Insert Here]</i>
Throughput (Requests / Second)	<i>[Insert Here]</i>	<i>[Insert Here]</i>
Error Rate Under Load	<i>[Insert Here]</i>	<i>[Insert Here]</i>
System Stability / Recovery Behavior	<i>[Insert Here]</i>	<i>[Insert Here]</i>

7.3 Testing Tools & Environment

- Load Testing Tool (e.g., JMeter, k6, Locust): *[Insert Here]*
- Test Environment Specification: *[Insert Here]*
- Test Data Strategy: *[Insert Here]*

8. Challenges & Learnings

This section documents the significant technical and operational challenges encountered during the development of NeoBank 360, along with the corresponding resolutions and key learnings derived by the team. It is intended to provide future contributors and stakeholders with institutional context that is not otherwise captured in code or test artifacts.

8.1 Technical Challenges Encountered

- Challenge 1: *[Insert Here]*
- Challenge 2: *[Insert Here]*
- Challenge 3: *[Insert Here]*

8.2 Key Learnings

- Learning 1: *[Insert Here]*
- Learning 2: *[Insert Here]*
- Learning 3: *[Insert Here]*

8.3 Recommendations for Future Enhancements

- Recommendation 1: *[Insert Here]*
- Recommendation 2: *[Insert Here]*
- Recommendation 3: *[Insert Here]*

Annexure A: Glossary of Terms

Term	Definition
JWT	JSON Web Token – a compact, signed token format used for authentication.
RBAC	Role-Based Access Control – an authorization model that restricts access based on assigned user roles.
N-Tier Architecture	An architectural pattern that separates an application into logical layers such as presentation, business logic, and data access.
ER Diagram	Entity-Relationship Diagram – a visual representation of data entities and their relationships.
<i>[Insert Here]</i>	<i>[Insert Here]</i>

Document Sign-Off

By signing below, the undersigned acknowledge that the contents of this handover document have been reviewed and accepted as an accurate representation of the NeoBank 360 system as of the date of submission.

Role	Name	Designation	Signature / Date
<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>
<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>
<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>	<i>[Insert Here]</i>